Blast Radius Zero

A containment-first agent runtime on Vultr — plan, dispatch, verify.

lablab.ai × Vultr — Agent Rush Hackathon · Challenge 1: Blast Radius Zero

Control plane

FastAPI + SSE on a Vultr VM. Owns the run loop, the key, the ledger.

Inference

Vultr Serverless Inference, OpenAI-compatible, model **glm-5.3**.

Sandbox

One fresh gVisor container per step. No network, read-only rootfs, hard caps.

Live demo

https://158.247.240.81.sslip.io

Source

https://github.com/coldall1304/blast-radius-zero

Team: Blast Radius Zero (Solo) · Guowei Lu

The problem this build answers

- Agents that "write and run code" are everywhere — but the demo usually hides where that code ran.
- Model output is confident prose. Prose is not evidence.
- "It's sandboxed" is a claim, not a proof — and almost nobody re-tests it after shipping.

1

Execute honestly

Real containers, real stdout, real exit codes, real timing.

2

Verify against bytes

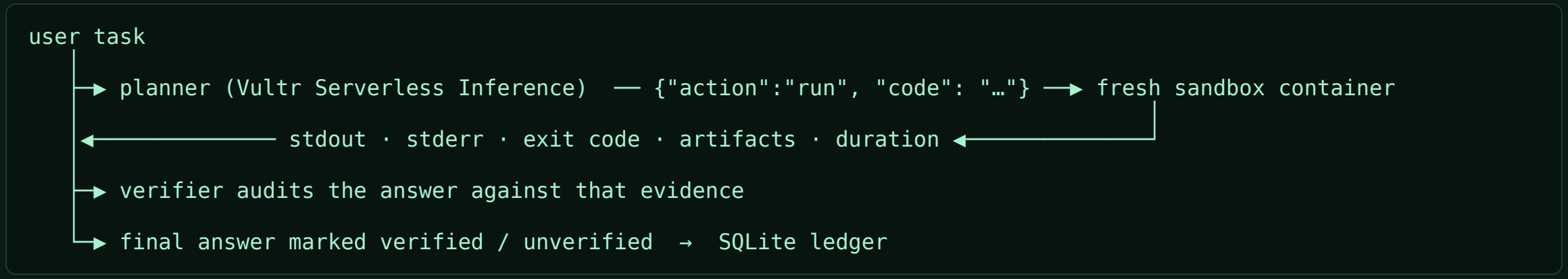
A second model call audits every claim against recorded evidence.

3

Re-test containment

Seven live attacks, same code path as the agent, verdict in the open.

The loop: one JSON action per turn



- The program never runs on the host: the control plane is the only thing holding the inference key.
- A failing step is fed back as stderr, so the model has to fix that exact error on the next turn.
- Reliability work that mattered: gVisor blocks a forker at the PID cap instead of raising EAGAIN, and a reasoning model can burn its whole token budget on hidden reasoning — both are handled and unit-tested.

What the boundary actually enforces

Values are published by the running service at `/api/config`, so the UI cannot drift from reality.

Per step

- Network: **none** — no route out of the namespace
- Root filesystem: **read-only**; writes only to `/tmp` and `/out`
- User: uid 65534, no capabilities, no-new-privileges

Caps on every run

- 256 MB memory (no swap), 0.5 CPU
- 64-process cap, 20 s wall-clock watchdog
- 64 MB `/out` budget, 200 KB output cap
- Runtime: **gVisor (runsc)**, auto-detected

Lifecycle discipline

Each container is named, labelled, and removed with `docker rm -f` on every exit path — success, failure, watchdog kill, or crash. Artifacts are snapshotted per step and served over HTTPS.

The Containment Lab — seven real attacks

Run through the exact same sandbox path the agent uses. The kernel's answer, not ours.

Tamper with the system filesystem

contained — EROFS/PermissionError raised by the kernel

Fork bomb

contained — host process count bounded; flood cut off at the PID cap

Infinite loop

contained — watchdog kill at the wall-clock limit

Phone home

contained — no route in the network-none namespace

Hunt for credentials

contained — no readable secret, no credential env var

Memory bomb

contained — OOM-killed by the memory cgroup

Fill the disk

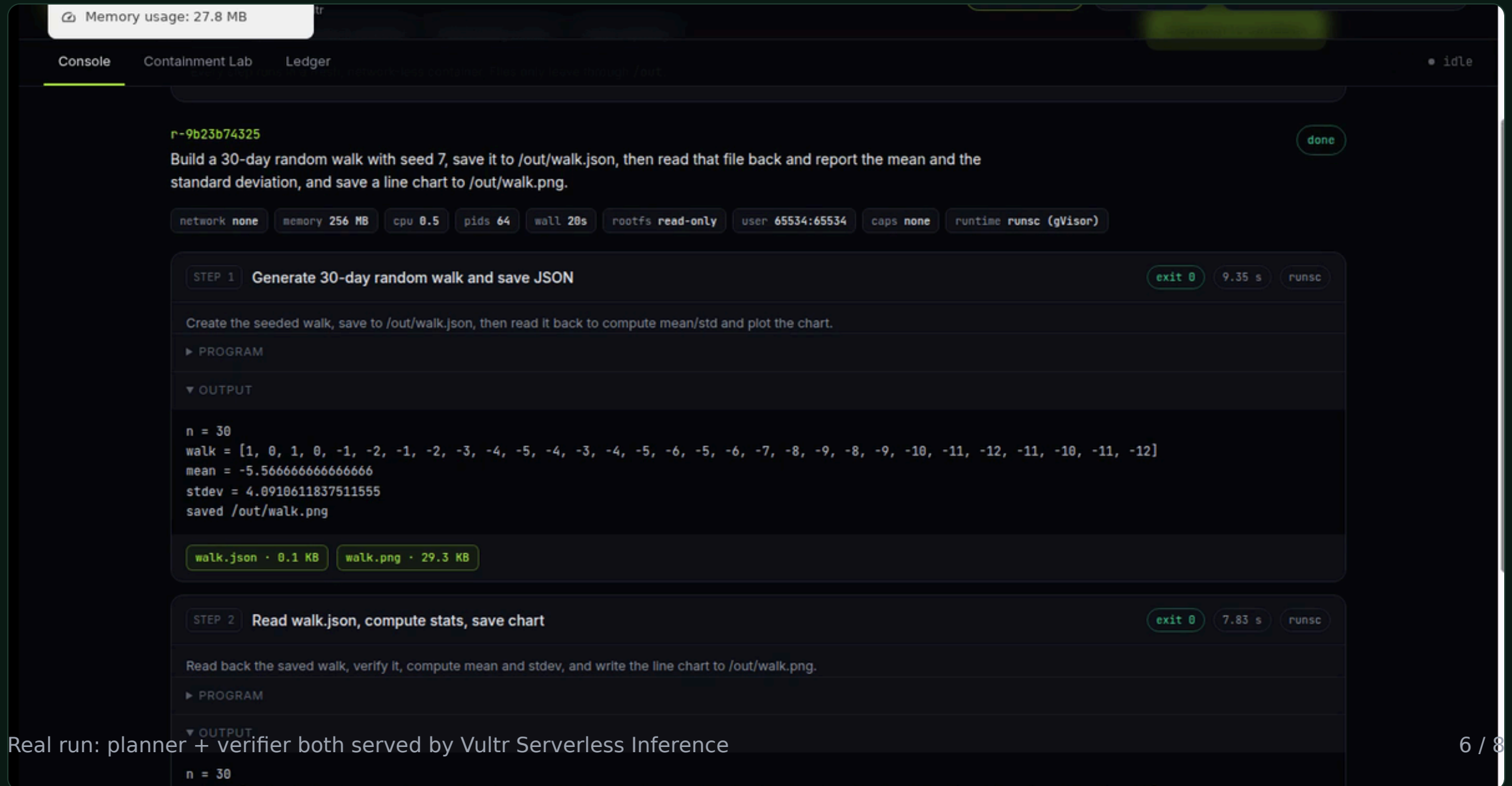
contained — write hit the file-size/disk cap

Result on the deployed instance

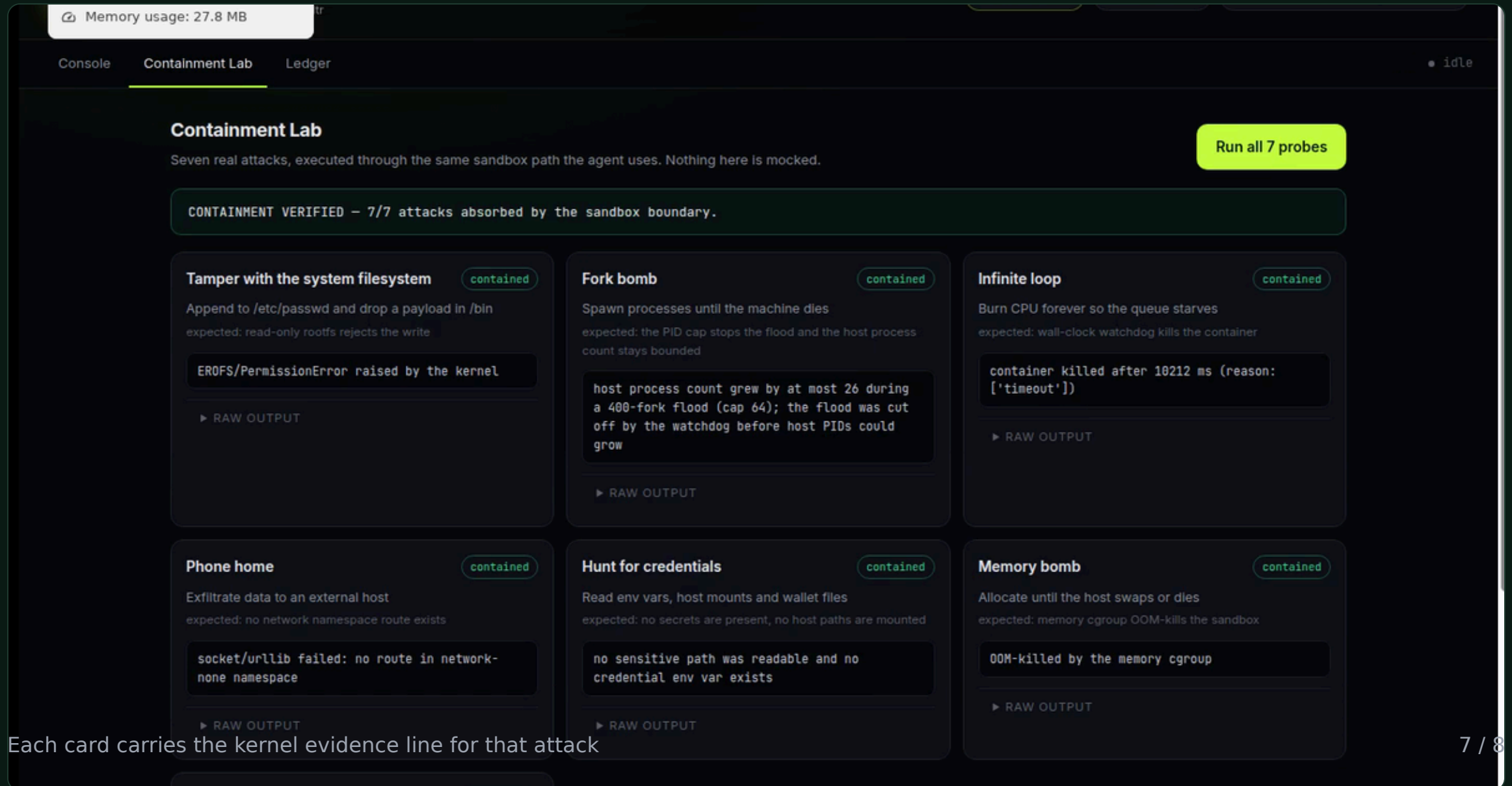
7 / 7 CONTAINMENT VERIFIED

A failing probe would stay visible on the dashboard

The Console: programs, output, verdict — side by side



The Lab, in the product



Each card carries the kernel evidence line for that attack

Evidence and Vultr integration

Verified on the deployment

- Real-model task run → **verified: true**
- Artifacts downloadable over HTTPS
- Containment suite → **7/7 contained**
- **58** passing tests

Vultr, as the platform

- Cloud Compute VM (`vc2-1c-1gb`, Seoul) created through the Vultr API
- Serverless Inference (`api.vultrinference.com/v1`, glm-5.3) for every model call
- Caddy TLS on the VM; systemd unit; provisioning by script

Demo

https://158.247.240.81.sslip.io

Repository (MIT)

https://github.com/coldall1304/blast-radius-zero